

The new server and backend structure

The 24 September plan found a careful codebase with no way to ship itself. Two weeks later every one of its eight phases is done and live: the server says what it runs, every change reaches it through one recorded and reversible command, the checks that guard a ride run on every change in both countries, and a backup can be put back in about a minute.

Verdict. All 31 steps of the plan are done. One of them (2.2) placed the SQL differently than planned, for a stated reason. Of the plan's 6 decisions, 4 are applied as written and 1 differently (6: releases are pushed, not merged). The last (1: a repository of its own) is deferred until after launch, as the plan itself recommended. Section 02 explains each difference. Fifteen releases went out, each on the owner's explicit OK, and none caused an incident. The server matches git byte for byte.

Repository	github.com/nammayatri-algeria/nammayatri, mirrored at github.com/MohaGNPro/nammayatri · branch <code>algeria/osrm-routing</code> , the default branch of both since 8 October
Server	Release M, commit <code>7f1f8e2e8e</code> . Every later commit changes documentation and tests only.
Period	Phases 0–7: 4–8 October 2026, plus three follow-up items on 8 October
Method	Every claim here was measured on 8 October, read-only, on the server, in git and through the GitHub API. Each phase's full record, with times, commits and outputs, is in <code>Backend/dev/local-stack/docs/box-snapshot-2026-10-04.md</code> .
Scope	Our deployment: <code>Backend/dev/local-stack</code> , the patch script and our workflows, and the server they run on. The mobile app and the website have their own plans.

Summary

What the plan found, and what is true now.

8 / 8

phases done and live

15

releases, one command each, each with the owner's OK

95 / 95

deployed files identical to git, checked from outside

22

test files on every push, plus a priced ride in each country

71 s

full restore, rehearsed from the offsite backup

The six things that mattered most, then and now

24 SEPTEMBER

8 OCTOBER

Nothing recorded what is deployed. The provenance of a service was a file's modification time.

`.shipped` names the commit, time, author and check result, and `.shipped.files` holds the sha256 of every shipped file. After every release, `ops/deploy.sh verify` recomputes the expected hashes from git on the laptop, so even a forged record is caught.

Six tracked files had drifted from the server, and copying git over the server would have removed two lines it needed.

0 differences. Git was corrected to match the server first (phase 1, 94/94), and since phase 3 the server only receives commits. `ops/deploy.sh status` reports any hand edit, and a release refuses to overwrite one.

The nightly backup ran a script in no repository (`/root/backup.sh`).

The backup script and its systemd units are in git and installed by the release; the unit runs the shipped copy. The offsite copy uses our own Google Drive client. Putting a backup back is scripted (`restore.sh`) and was rehearsed: 71 s, 106 tables exact.

The ride regression had been red since 20 September, and it ran so rarely that nobody saw it.

Green since phase 4. It runs on every push and every Monday, on the server's exact image, with the live two-country layout. It prices a ride in Algiers and in Nouakchott, each priced only by its own country's merchant.

35 probes sat beside the scripts a deploy runs.

The probes are kept in `investigations/`, and the retired account scripts in `retired/`. Only `stack/` is deployed, and the server holds no probe.

One 3,802-line README, 258 lines of it about a billing model retired on 7 September, whose 694 lines of code were still live.

The README is 236 lines plus 15 pages in `docs/`, moved word for word (proven line by line), with 5 decision records and 2 runbooks. The old subscription is gone: routes, code and README section. Its tables were dumped, encrypted, then dropped.

Also found and fixed on the way:

- a Redis publish that never settled;
- CI testing a six-week-old backend image instead of the server's;
- 58,329 reviewed Arabic place names that were in no backup;
- a restore that could start loading during Postgres's first-run restart;
- a release command that would have restarted the maps service twice.

Each was found by a check written during the restructuring, before it reached a user.

The plan, step by step

Every step of the 24 September plan, with what proves it.

#	STEP	STATUS	EVIDENCE, 8 OCTOBER
Phase 0 · Photograph the machine			
0.1	Tag <code>pre-restructure</code> and push it	DONE	The tags <code>pre-restructure</code> and <code>snapshot-2026-10-04</code> are on both remotes.
0.2	Full read-only inventory of the server, committed	DONE	<code>docs/box-snapshot-2026-10-04.md</code> and <code>docs/box-inventory-2026-10-06.md</code> (every file, by hash)
0.3	Record the running binaries; re-create <code>bin/MANIFEST.txt</code>	DONE	<code>bin/MANIFEST.txt</code> is present on the server. The running backend is the image <code>ny-backend@108eca6c</code> , measured by hash.
0.4	Copy <code>/root/backup.sh</code> into git, unchanged	DONE	It's now <code>stack/backup.sh</code> , which the nightly unit runs.
Phase 1 · Make git match the box			
1.1– 1.4	Settle each drifted file; take the box's compose lines; one <code>backup.sh</code> ; refresh the compose warning	DONE	94/94 identical on 6 October; 95/95 today. The compose file carries the <code>edge-web</code> mount and the <code>movin-driver-docs</code> volume.
Phase 2 · Sort the folder			
2.1	Move the 35 probes into <code>investigations/</code>	DONE	<code>investigations/</code> holds 36 files; the server holds 0 probes.
2.2	Separate what is deployed, the SQL, and what a release runs	DIFFERS	<code>stack/</code> and <code>ops/</code> as planned. The SQL is in <code>stack/db/</code> , not <code>local-stack/db/</code> : the apply scripts run on the server, so the SQL must ship.
2.3	Revert the four <code>Frontend/android-native</code> edits	DONE	Commit <code>117ad97b03</code>
2.4	Upstream's workflows off our branches	DONE	The last 30 runs on <code>algeria/osrm-routing</code> , on both repositories, are all ours. HLint skips <code>algeria/*</code> , and <code>stale</code> is disabled.
Phase 3 · Turn the deployment into a release			
3.1	<code>ops/deploy.sh</code> : refuse a dirty tree, ship, apply SQL, restart only what changed, check, write <code>.shipped</code>	DONE	15 releases. Modes: <code>--dry-run</code> , <code>status</code> , <code>verify</code> , <code>rollback</code> , <code>tidy</code> .
3.2	Previous tree kept as <code>local-stack.prev</code> , replacing the <code>.bak</code> files	DONE	35 leftover copies archived; 0 remain in the stack.
3.3	Compose overlay; the two repositories stop editing each other's file	DONE	<code>COMPOSE_FILE=docker-compose.yml:compose.admin.yml</code> . The website's console is the overlay, and its scripts now only check, never write.
3.4	First release deploys what is already there	DONE	<code>0d463c3016</code> : nothing restarted, every hash as released.
Phase 4 · The checks that guard a ride			
4.1	Diagnose and fix the ride regression	DONE	Three stacked causes, each read in the backend's log: no routing, a pipefail-killed poll, two missing columns. Green since run 22.
4.2	Run it on every push	DONE	Every push to <code>algeria/**</code> , plus Mondays 06:00 UTC since our branch became the default.
4.3	The backup unit runs the repository's copy; the release owns it	DONE	<code>ExecStart=/opt/ny/local-stack/backup.sh</code> . Units are in <code>stack/systemd/</code> , installed by the release.

4.4	Post-release hash check against the commit	DONE	ops/release-verify.py , run after every release. Proven to catch a forged record.
Phase 5 · The shims become software			
5.1	package.json and lockfile for each shim	DONE	npm ci from the lockfile. The pg pin left the Dockerfile.
5.2	Node 22, with a rollback image	DONE	Both shims run v22.23.3, and every replaced image is kept as <container>:previous .
5.3	Money-path tests: wallet, restricted, deletion	DONE	103 checks in a real Postgres (PGLite). 18 faults planted on purpose, all caught. They found one real bug.
5.4	Split both server.js by subject, no behaviour change	DONE	maps-shim 886 → 391 lines, 14 modules; the guard 1,782 → 1,119 lines, 9 modules. Golden-file tests replay 82 requests, so every answer is proven unchanged.
Phase 6 · Retire the old billing model			
6.1	Measure who calls /subscription/*; retire the routes	DONE	No phone had called it since 2 September. It now answers 410 (release J).
6.2	Dump, then drop, the three objects	DONE	An encrypted dump, local and offsite, checked by row count and decryption; then dropped (release K). The SQL refuses if anything wrote after 1 September.
6.3	Delete subscription.js and the 258-line section; a note in the wallet page	DONE	Both gone. docs/wallet.md → What it replaced
Phase 7 · Documentation that stays true			
7.1	Short README plus docs/ , one page per subject	DONE	4,127 → 236 lines plus 15 pages. A script proves all 3,263 lines kept, and a CI test checks every link.
7.2	ADRs: shims, config, patches, merchants, wallet	DONE	docs/adr/0001-0005
7.3	Release and rollback runbooks	DONE	docs/runbooks/release.md , rollback.md
7.4	What is ours and what is upstream	DONE	docs/ours-and-upstream.md

The six decisions

#	QUESTION	STATUS	WHAT WAS DONE
1	Stack in its own repository?	AFTER LAUNCH	The plan said "Move it, after launch". Not yet launched, so deferred. The stack/ boundary makes the move a copy.
2	Compose overlays?	DONE	The website's console is an overlay; neither repository writes the other's file.
3	package.json and Node 22?	DONE	Phase 5, with rollback images.
4	Keep the probes?	DONE	Kept, moved, outside the release.
5	Retire /subscription/* , drop the tables?	DONE	Phase 6, in that order.
6	Ride regression blocks a merge?	DIFFERS	Releases are pushed directly, not merged, so there is no merge to block. The release runbook requires both workflows green before a release.

The target architecture also named bin/MANIFEST.txt "written by the release". The binaries now run from an image, not from bin/ , and the image's digest is measured, not yet recorded in .shipped . That's listed in section 10.

The server today

One server, 21 containers, three systemd services. Nothing on it is edited by hand.

FROM THE INTERNET

edge · nginx

TLS for the API, the console and the public site · rate limits per zone · the OTP attempt limit · size limits on every body ·
/subscription/ answers 410



OURS — THE RULES OF THIS DEPLOYMENT

auth-guard

Sign-in codes by SMS (Moorsyl), WhatsApp, or an SMS the person sends · countries open to sign-in · attempt and spend limits · the WALLET_EMPTY gate · rating reports. Node 22, 9 modules.

maps-shim

"Google" for the backend: routes (OSRM), place search, reverse geocoding · the driver wallet and top-ups · who may work (the dispatch list) · the iPhone push relay · avatars, ratings, account deletion requests, number change. Node 22, 14 modules.

admin console API

The website's service, an overlay compose file of its own: drivers, documents, wallets, messages, the server screen.



UPSTREAM NAMMA YATRI, 2023, PLUS 54 PATCHES APPLIED AT BUILD TIME — ONE IMAGE

rider-app

The passenger side (BAP). One merchant, both countries.

beckn-gateway

Routes every search to the driver merchants.

mock-registry

Who the providers are, and where they live.

driver-app

The driver side (BPP). One merchant per country; prices, dispatch.

MAPS

OSRM

Routing on the combined Algeria + Mauritania map

tileserver-gl

The map picture, same extract

service-area map

What the backend answers, anywhere you click

DATA

Postgres + PostGIS

atlas_app · atlas_driver_offer_bpp · atlas_registry · movin (wallets, papers, console) · geo (121,886 places)

passetto

Encrypts phone numbers; its own database

Redis

Caches, positions, the dispatch list

Kafka

The backend's events

SYSTEMD, BESIDE THE CONTAINERS

movin-backup

Nightly 02:30, encrypted, offsite; runs the shipped backup.sh

movin-bot

The owner's reports, both countries

movin-fleet

12 simulated cars while the launch waits — removed before launch

The principles the structure now encodes

- **Only what a release ships lives in** `stack/`. If it isn't deployed, it isn't in there.
- **A release is a command.** It ships a commit, records it, and keeps the previous version under a name.
- **The server can answer "what are you running?"** through `.shipped`, and the laptop can check that answer.
- **Neither repository edits the other's file.**
- **New behaviour goes beside the backend**, as config, SQL or a shim, and the Haskell is patched only when nothing else reaches it.
- **Nothing is deployed without the owner saying so**, one OK per release.

The repository

About 210 of 15,381 files are ours. The rest is upstream's, which we do not edit.

```
Backend/dev/local-stack/                202 files · 37,181 lines
├─ stack/                               WHAT THE SERVER RUNS – the only folder a release ships    95 files
│   ├── docker-compose.yml             the stack (the console is its overlay)
│   ├── auth-guard/                   server.js (the sign-in flow, 1,119 lines) + 8 modules
│   ├── maps-shim/                   server.js (the router, 391 lines) + 13 modules
│   ├── edge/                         nginx: names, TLS, rate limits
│   ├── db/                          every .sql – tariffs, geofences, merchants, wallet, retirements
│   ├── geocoder/                   the place index and its Arabic names
│   ├── systemd/                   the backup units a release installs
│   ├── backup.sh                   the nightly backup                restore.sh    put one back / rehearse
│   ├── setup.sh                   a stack from nothing – and the two-country layout CI uses
│   └─ simulate-driver.py · movin-bot.py · apply-*.sh · *-prepare.sh
├─ ops/                               RUN FROM THE LAPTOP                                6 files
│   ├── deploy.sh                   THE way to change the server
│   ├── release-remote.py           its server half                release-verify.py  the outside check
│   └─ checks/                     the two-country ride probe
├─ tests/                          22 test files, all run in CI                                31 files
├─ docs/                          15 subject pages · adr/ · runbooks/ · records of the server  28 files
├─ investigations/                the probes: how the system was understood (not deployed)    36 files
├─ retired/                       the scripts that made test accounts (dev stacks only)       5 files
└─ README.md                      236 lines: what this is, where everything is, your first change

.github/workflows/algeria-*.yaml     node tests · ride regression · backend build
.github/scripts/algeria/              apply-patches.py – 54 patches to upstream 03a7531
CLAUDE.md                            the fork's rules, from "This fork: Movin" down
```

What is not ours

`Backend/app` and `Backend/lib` are upstream's Haskell, about 10,800 commits newer than what runs. The server runs upstream 03a7531 (March 2023) with 54 patches applied during the CI build, and the tree in this repository is never edited. `Frontend/` is upstream's PureScript app, which we do not build. It has been identical to upstream since phase 2. The passenger and driver app (React Native) and the website are separate repositories.

How a change reaches the server

One command, run from the laptop, with a dry run and a way back.

STEP	WHAT HAPPENS
Before	Tests pass locally; the commit is pushed; both workflows are green on it; <code>ops/deploy.sh --dry-run</code> lists every file, the SQL and the restarts; the owner gives an OK for this release.
Refusals	A dirty working tree, an unpushed commit, or a file edited by hand on the server since the last release: each one stops the release and names the cause.
Ship	<code>stack/</code> is taken from the commit (<code>git archive</code>) and written in place, so bind-mounted files keep their inode. Nothing outside git is touched: <code>.env</code> , secrets, certificates, data. At most four SSH connections; the firewall refuses a sixth within 30 s.
Keep the way back	Every replaced file goes to <code>local-stack.prev</code> , and every replaced image is tagged <code><container>:previous</code> .
SQL	A <code>db/*.sql</code> is applied only when its content is new to the server. Re-running a tariff would undo every fare changed since.
Restart only what changed	Guard code → the guard · maps-shim code → maps-shim · <code>nginx.conf</code> → <code>nginx -t</code> then reload · compose → only the services whose resolved config changed, and never restarted twice · systemd units → installed and reloaded
Check	Every container still running, both health checks 200 (with a 30 s wait), <code>nginx -t</code> , every hash; then <code>.shipped</code> is written.
Verify from outside	The server only measures; the laptop recomputes the expected hashes from git. A wrong commit or a forged record fails.
Undo	<code>ops/deploy.sh rollback</code> restores the files (one level); <code>:previous</code> restores an image; a dump or <code>restore.sh</code> restores data. Rollback never reverses SQL.

`tests/release.test.sh` rehearses all of this on a fake server in CI: a release, a refused hand edit, the outside check including a forged record, the units, a rollback, a tidy and the single restart.

The fifteen releases

RELEASE	COMMIT	DATE	WHAT IT SHIPPED	RESTARTED
first	<code>0d463c3016</code>	6 Oct	The new layout (<code>db/</code>); 44 old files out	nothing
phase 4	<code>37c3d6f04e</code>	6 Oct	Backup units owned by the release; <code>setup.sh</code> fixes	nothing
A	<code>64a94a4508</code>	6 Oct	Shim packaging; the Redis publish fix	maps-shim (rebuilt)
B	<code>491087ce2d</code>	6 Oct	Node 22 for both shims	both shims
C, D	<code>b5999a1c06</code> , <code>c08aa72e8a</code>	6 Oct	maps-shim split: directions, places	maps-shim
E–I	<code>714bb68c7e</code> ... <code>9f8ff3b09c</code>	6 Oct	Guard split: driver rules, limits, gateway, personal codes, number change	the guard, once each
J	<code>fe4a44c3a8</code>	7 Oct	Subscription routes retired (410), its code removed	maps-shim (once), edge reload
K	<code>a1582735cd</code>	7 Oct	Dump, then drop the subscription tables	nothing
L	<code>1548e998e7</code>	8 Oct	<code>restore.sh</code> ; Arabic names in every backup	nothing
M	<code>7f1f8e2e8e</code>	8 Oct	Restore timing fix; the two-country CI steps	nothing

Every release got its own explicit OK, after a dry run and a plain statement of what would restart and who would notice. One release (C) recorded a false alarm, a health check taken one second after a restart. The checker now waits up to 30 s.

The checks

A red tick now means something specific, and it appears the same day.

On every push: `algeria: node tests` (Node 20 and 22)

AREA	TEST FILES	WHAT THEY HOLD
Money: who may work	wallet, restricted, wallet-dispatch, deletion, auth-guard-wallet-gate	The prepaid day, top-ups, the credit race, the dispatch list, the 403. SQL runs in a real Postgres (PGLite).
Sign-in	auth-guard-limits, -signup, -sms-in, -sms-inbox, -trusted, -whatsapp, -number-change, number-change	Codes, limits, the countries open to sign-in, WhatsApp, an SMS the person sends, trusted phones, number change.
Behaviour pinned	maps-shim-routes, auth-guard-routes	Golden files: 32 + 50 requests, every answer, query and outgoing call. A refactor must pass them untouched.
Releases and data	release, restore, retire-subscription	The release on a fake server; a full backup and restore (rehearse, refuse when broken, live) with invented data; the drop SQL's guard.
Other	push-relay, driver-push, movin-bot, docs-links	iPhone push, office push, the bot's queries, every documentation link.

On every push and every Monday: `algeria: ride regression`

The test starts a complete backend from nothing on a throwaway machine, using **the server's exact image**. It routes on Algiers and Nouakchott cut from the country maps, and applies the server's own two-country SQL. Then a `+213` rider in Algiers and a `+222` rider in Nouakchott each ask for a ride. Every estimate must come from that country's driver merchant: in the first green run, 629 / 899 DA in Algiers and 74 / 74 / 148 MRU in Nouakchott.

Found by this check on 8 October: until then CI pulled `ghcr.io/<repository owner>/ny-backend`, which in the owner's repository is a 26 August build, six weeks older than the server's and unable to sign in a Mauritanian number. It now pulls the image `deploy-backend.sh` deploys, and prints the digest it tested.

How the tests themselves were tested

Each new test was shown able to fail by breaking the code on purpose: 18 faults for the money path, 4 for each golden file, the double-restart rule, the split proof, the link check, and the restore's single transaction. For the restore, the fault was pushed to a throwaway branch so that CI itself caught it.

Backups, and putting one back

A backup nobody has restored is a guess. This one has been restored.

WHAT	HOW
Contents	The data schemas (riders, drivers, rides, wallets, papers, the console), passetto's keys (without them the phone numbers are unreadable), the drivers' sign-in codes, the driver documents, and since 8 October the 58,329 Arabic place names, keyed by the OSM id that a rebuild keeps. The place index itself is rebuilt, not restored.
When and where	Nightly at 02:30, encrypted (GPG AES-256) on the server, then copied to Google Drive through our own client. Older copies are kept as 7 daily and 4 weekly. A changed set of database schemas stops the backup instead of silently leaving one out.
Check	<code>./backup.sh restore F</code> loads a backup into a scratch database and compares counts.
Rehearse	<code>./restore.sh rehearse offsite:latest</code> fetches from Drive, restores into a throwaway copy (its own private network, no ports), checks every part, and removes it.
Put back	<code>./restore.sh live F</code> : • refuses if anything outside the data depends on it; • asks for a typed confirmation, then takes a safety backup; • stops what reads the data, then loads the data and the keys, each in one transaction, so a broken archive changes nothing; • flushes the caches, restarts everything and checks.

The rehearsal, 8 October, from the offsite copy

71 s fetch to removal	106 tables, each with exactly the rows of the dump	20 / 20 sampled phone numbers decrypted on the restored keys	58,329 Arabic names matched	25 / 25 driver documents back, and opening
---------------------------------	---	--	---------------------------------------	---

The 21 live containers ran throughout, and nothing was left behind. Not yet rehearsed: a whole new server, which means bringing the stack up and rebuilding the place index before `restore.sh live`. The rollback runbook says so.

08

Two countries on one stack

Mauritania and Algeria, since 13 September.

SIDE	MERCHANT	SERVES	PRICES
Passenger	YATRI	Mauritania and Algeria	none
Driver	favorit0-...	Mauritania	MRU, mauritania-tariff.sql
Driver	algeria0-... (a clone of the proven one)	Algeria	DA, algeria-tariff.sql

Every per-merchant statement names its merchant: an unkeyed tariff would reprice the other country. Geofences exist on both sides and must agree. The CI ride regression now applies these same files and checks each country is priced only by its own merchant.

09

The decisions behind it

Written down as records in docs/adr/.

DECISION	IN ONE LINE
0001 · Shims, not Haskell	New behaviour goes beside the backend: config first, then a shim, then nginx; a Haskell patch only when nothing else reaches.
0002 · Config, not code	Service endpoints are database rows. Pointing googleMapsUrl at maps-shim is the whole maps integration.
0003 · Patches at build time	The running Haskell is upstream 03a7531 plus 54 patches across 29 files, applied in CI; the tree here is never edited.
0004 · One merchant per country	Each country is its own driver merchant; the passenger side keeps one.
0005 · The wallet	No top-up, no work: a day's price comes off at the first ride, enforced in dispatch, the guard and the app.

What remains

None of it blocks anything. Each is a separate decision.

ITEM	WHEN	WHY
A security hardening step for stored personal data	Before the first real passenger	Planned with the owner as a separate final step: rehearsed on a copy first, then a short maintenance window. Its details are tracked privately, not in this public repository.
Remove the simulated fleet and the two test passengers	Before launch	The 12 simulated cars answer real requests.
Record the backend image in <code>.shipped</code>	Any release	Today the image is measured by its digest; recording it would let <code>verify</code> check the binaries too.
Move the stack to its own repository	After launch	Plan decision 1. The <code>stack/</code> boundary makes it a copy.
Rehearse a restore onto a brand-new server	Before it is needed	The restore itself is proven; bringing up an empty server around it is not.
A newcomer follows <i>Your first change</i>	When someone joins	Phase 7's criterion is met on paper; it is proven only when someone does it.
App restructuring phases 4–8, with tap-through UI tests	Next	The app's own plan.

A Appendix • measurements

24 September (the plan's audit) against 8 October.

MEASURE	24 SEPTEMBER	8 OCTOBER
Records on the server of what is deployed	0	<code>.shipped</code> + 95 hashes
Deployed files that differ from git	6	0 (95/95)
<code>.bak</code> copies on the server	30 (35 when tidied)	0
Backup script under version control	no	yes, and installed by the release
Backup restore	scratch check only	scripted, rehearsed: 71 s
Ride regression	2 runs, both red	green on every push and weekly, both countries
Test files run in CI	4 (570 lines)	22 (6,301 lines with fixtures)
Node version in the shims	20 (end of life)	22.23.3, previous image kept
<code>maps-shim/server.js</code> · <code>auth-guard/server.js</code>	803 · 1,273 lines	391 · 1,119 lines (14 and 9 modules)
README	3,802 lines	236 lines plus 15 pages
Retired billing code live	694 lines	0
Files that are ours	133 of 15,305	about 210 of 15,381
Containers on the server	21	21
Disk used	41 G of 193 G	60 G of 193 G
Secrets in git	0	0

Measured read-only on 8 October 2026: on the server (`.shipped` , hashes, units, containers), in git, and through the GitHub API. Line counts include comments and blank lines. The 24 September figures are the plan's own.